



TEMA II

DESARROLLO DE APLICACIONES BACK EN PHP

*"Un sueño no se hace realidad por arte de magia,
necesita sudor, determinación y trabajo duro"*
Colin Powell

2. FUNDAMENTOS DEL LENGUAJE

- El final de cada instrucción PHP se designa con un punto y coma (;).
- Los bloques de instrucciones correspondientes a bucles, sentencias condicionales, y funciones deben ir entre llaves {}.
- El intérprete PHP ignora los espacios, los tabuladores y los saltos de línea; podemos poner tantos espacios, tabuladores y saltos de línea como queramos para mejorar la legibilidad de nuestro código porque el intérprete considerará que todo lo que escribamos corresponde a una misma instrucción hasta que encuentre un punto y coma;.
- PHP es parcialmente insensible al uso de mayúsculas y minúsculas, pero no totalmente insensible. Al intérprete PHP le da igual que escribamos los nombres de las instrucciones y de las funciones en mayúsculas o minúsculas o mezclando mayúsculas o minúsculas; por ejemplo, echo funciona igual de bien que ECHO o eChO. Sin embargo, el intérprete PHP es mucho más exigente con los nombres de variables y constantes; en este caso debemos ser coherentes y escribirlos siempre del mismo modo. Una norma no escrita aconseja utilizar mayúsculas para los nombres de las constantes y minúsculas para los nombres de las variables.
- // sirve para crear comentarios de una sola línea.

Ejemplo 1: "Mi primer código en php"

```
<html>
<body>
<?php
    $myvar = "Hola. Este es mi primer script en PHP \n";
    //Esto es un comentario es mi primer script en PHP \n";
    echo $myvar;
?>
</body>
</html>
```

Variables

En PHP todas las variables deben empezar con el signo \$, seguido de una letra o un signo de subrayado, seguido de cualquier combinación de letras, números y signos de subrayado.

PHP es sensible al uso de mayúsculas/minúsculas en las variables; por ejemplo, \$nombre y \$Nombre se interpretan como dos variables diferentes.

No nos obliga a especificar el tipo de datos. Una misma variable puede contener un número en un punto del script y una cadena de caracteres en otro punto diferente. Cuando definimos una variable asignándole un valor, le atribuye un tipo. Si por ejemplo definimos una variable entre comillas, la variable será considerada de tipo cadena:

```
$variable="5"; //esto es una cadena
```

Sin embargo si pedimos en nuestro script realizar una operación matemática con esta variable, no obtendremos un mensaje de error sino que la variable cadena se convierte en numérica

```
<?php
$cadena="5"; //esto es una cadena
$entero=3; //esto es un entero
echo $cadena+$entero;
?>
```

Por defecto PHP asigna los datos a las variables por valor; esto quiere decir que si asignamos a una variable otra variable, aunque esta segunda cambiasse de dato no cambiaría el dato de la primera.

```
$a=7;
$b=$a;
echo $a;
$b=$b+8;
echo $a;
```

Cadenas de caracteres

En PHP un dato de tipo cadena de caracteres debe ir escrito (preferiblemente) entre comillas simples. Para incluir en una cadena de caracteres el carácter de delimitación ' tendremos que escribirlo precedido por el carácter \ (\). La anteposición del carácter \ a otro carácter se denomina escapado. Para incluir el carácter \ tendremos que escribirlo \\\

También se pueden usar comillas dobles, en este caso el intérprete busca dentro de la cadena nombres de variables y, si los encuentra, los sustituye por sus correspondientes valores.

Al usar comillas dobles, los caracteres que requieren escapado para poder mostrarse correctamente son \\, \" y \\$.

Si no necesitamos esta funcionalidad de sustitución de variables dentro de una cadena de caracteres es preferible usar comillas simples porque se interpretan más rápido.

Para concatenar dos cadenas de caracteres se utiliza el operador punto (.)

```
<?php
$cadena1='hola ';
$cadena2=' que tal';
echo "$cadena1"."$cadena2";
?>
```

Tipos de datos

- **Booleanos** o lógicos (bool): sólo pueden tener el valor TRUE o FALSE
- **Números enteros** (integer, int): Cualquier número que pertenezca al grupo de los enteros (no simplemente natural; es decir, cualquier número positivo o negativo incluido el cero), dentro de los límites de representación que permita la arquitectura del ordenador.
- **Números de coma flotante** (float,double): Cualquier número que pertenezca al grupo de los reales. En las versiones iniciales de PHP este tipo de datos se llamaba número doble.

- **Cadenas de caracteres** (string).
- **Matrices** o arrays (array).
- **NULL**.
- **Object**

PHP no permite definir el tipo de datos de las variables, ni nos obliga a almacenar siempre el mismo tipo de datos en una variable. Lo establece en función del dato que tenga asignado en cada instante la variable. Para poder operar con datos de distinto tipo, los convierte internamente a un mismo tipo aplicando ciertas reglas. Estas reglas dependen de cada operación concreta y de los tipos de los operandos.

PHP nos ofrece la función **var_dump** para obtener información sobre una variable, dato o expresión. Devuelve el tipo del dato y su valor.

Podemos forzarla conversión (casting). Se escribe entre paréntesis delante del dato, variable o expresión el tipo al que queremos forzarlo.

También podemos utilizar la función `setType()`:

```
setType($variable,"nuevo_tipo");
```

actualiza el tipo de `$variable` a "nuevo_tipo" y devuelve un booleano indicando si hubo éxito o no en la conversión.

Ejemplo 2: "Tipos de datos y variables"

```
<?php
$nombre='pepe';
$edad="20";
var_dump($nombre);
var_dump("pepe");
var_dump($numero+6);
$real=4.5;
$entera=(int)$real;
echo $entera;
setType($real, "int");
var_dump($real);
echo (float)$entera;
?>
```

Reglas automáticas de conversión de tipos de datos

Ya hemos comentado anteriormente que PHP nos permite combinar datos de distintos tipos en una misma operación. Esto es posible porque de forma interna y automática se lleva a cabo una conversión de los datos implicados en la operación a un tipo común. Esta conversión está regida por numerosas reglas, algunas muy específicas y difíciles de recordar, por lo que es preferible ser rigurosos a la hora de programar y utilizar en cada operación sólo datos del tipo que espera el operador.

Se comentan algunas de las reglas automáticas de conversión que sí conviene recordar porque pueden ahorrarnos escribir mucho código:

- En operaciones lógicas, los datos NULL, 0, '0' y '' se consideran FALSE. Cualquier otro dato se considera TRUE (incluida la cadena 'FALSE').
- En operaciones aritméticas no unitarias las cadenas se intentan leer como números y, si no se puede, se convierten en 0, TRUE se convierte en 1, y FALSE se convierte en 0.
- En operaciones de comparación, si un operando es un número, el otro también se convertirá en un número. Sólo si ambos operandos son cadenas se compararán como cadenas de caracteres.

Constantes

Las constantes son nombres que se refieren a un dato que no cambia a lo largo de la ejecución del script. Son reconocidas en el script en el que son definidas tanto dentro como fuera de las funciones. Son globales.

define(nombre_constante_entre_comillas,dato_constante)

```
<?php
define("PI", 3.14);
echo PI;
?>
```

Los nombres de las constantes no van precedidos del signo \$, **por convenio se escriben completamente en mayúsculas.**

Existe otra función, llamada **defined** que nos permite saber si se ha declarado una constante determinada, lo haremos de la siguiente manera:

```
<?php
$definida = defined("miVariable");
// Mostrara 0 si existe la variable y 1 si no existe
echo $definida;
?>
```

En algunos casos no es posible acceder al valor de una constante directamente por su nombre, pues puede coincidir con el de una función y el resultado producido no sería el esperado, en ese caso usaremos la función **constant** para leer el valor de dicha constante:

```
<?php
define ('echo','valor de mi constante');
echo constant('echo');
?>
```

PHP incluye algunas **constantes predefinidas**, y entre ellas destacan algunas denominadas popularmente *constantes mágicas*, cuyo valor es asignado por el propio intérprete y varía en función de dónde sean utilizadas (no son verdaderas constantes).

CONSTANTE	VALOR
__FILE__	Contiene la ruta completa del archivo en el que se encuentra el script
__DIR__	Contiene el nombre de la carpeta en la que se encuentra el archivo del script.
__LINE__	Contiene la línea que se está ejecutando en ese momento exacto del fichero actual
__FUNCTION__	Añadida en la versión 4.3 de PHP, contiene el nombre de la función que se está ejecutando actualmente
__CLASS__	- Añadida también en la versión 4.3, contiene el nombre de la clase que se ejecuta actualmente
__METHOD__	Añadida en la versión 5, contiene el nombre del método actual
PHP_INT_MAX	Contiene el máximo entero utilizable por el sistema.

```
<?php
echo __FILE__;
echo '<br>';
echo __DIR__;
echo PHP_INT_MAX;?>
```

Si al convertir una variable excedemos su máximo tendremos overflow

Si el resultado de una operación en la que esperamos un entero, supera ese límite máximo, se devolverá como un float.

Operadores básicos

Las operaciones no se realizarán de izquierda a derecha sino que se realizarán siguiendo un orden de preferencia que PHP asigna a cada operando, estas son las preferencias de PHP en cuanto a los operadores:

```
new
[
! ~ ++ -- (int) (float) (string) (array) (object) @
* / %
+ - .
<<>>
<<= >>=
== != === !==
&
^
|
&&
||
?:
= += -= *= /= .= %= &= |= ^= <<= >>=
and , or
```

- Aritméticos :
 - +: Suma
 - -: Resta
 - *: Multiplicación
 - /: División (float)
 - %: módulo
 - ++: Incremento unitario
 - --: Decremento unitario
 - +=, -=, *=, /=
- De comparación:
 - == : Igualdad (compara valor y no tipo)
 - ===: idéntico (compara valor y tipo)
 - !=, <>: Desigualdad
 - !==: no idénticos
 - <: Menor que
 - >: Mayor que
 - <=: Menor o igual que
 - >=: Mayor o igual que

- Lógicos :
 - !: negación
 - and, &&: Y
 - or, |: O
- Operador asignación =
- Operador concatenación .
- (.=), este operador concatena a una variable un valor dado de la siguiente manera

```
<?php
$html = "<html>";
$html .= "<head>";
$html .= "<title>Mi página</title>";
$html .= "</head>";
$html .= "<body>";
$html .= "Este es el contenido de mi";
$html .= " página";
$html .= "</body>";
$html .= "</html>";
echo $html;
?>
```

Arrays

Los datos no están obligados a ser todos del mismo tipo, podemos mezclar datos numéricos con cadenas de caracteres, por ejemplo.

Para indicar que una variable va a referirse a un array debemos usar el constructor array.

La asignación de datos a una variable de tipo array puede hacerse en la propia declaración:

```
<?php
$lista=array('pepe', 'juan',45);
?>
```

O de forma individual mediante la utilización de la sintaxis de corchete vacío:

```
<?php
$lista=array('pepe', 'juan',45);
$lista[]=8;    //añade un nuevo elemento al final con valor 8 ?>
```

Los elementos de una array vienen identificados por un número o por una clave que se define así

```
<?php
$vector = array ( "clave 1" => "elemento 1", "clave 2" => "elemento 2" );
?>
```

```
<?php
$persona=array('edad'=>38,'peso'=>80,'pelo'=>'moreno');
echo $persona['edad'];
echo '<br />';
echo $persona['pelo'];
?>
```

La sintaxis de corchete sirve para referirse a los elementos que componen el array, teniendo en cuenta que cada uno de ellos está asociado a un índice numérico (el índice del primer elemento es el 0) o la clave con la que fue definido. Si definimos claves no podemos usar el índice numérico para referirnos a los elementos.

Si usamos una clave que no existe, estaremos creando un nuevo elemento (con la clave que hayamos usado).

```
<?php
$vector = array ( "elemento 1", "elemento 2" ); // Añadimos "elemento 3" al final
$vector[] = "elemento 3"; // Añadimos un elemento con clave "clave 3"
$vector['clave 3'] = "elemento 4";
$vector[2] = "elemento N";
print_r ($vector);

/* Array
(
    [0] => elemento 1
    [1] => elemento 2
    [2] => elemento N
    [clave 3] =>elemento 4
) */ ?>
```

No podemos usar la instrucción `echo` para imprimir todos los elementos de un array, pero sí la instrucción **`print_r`**.

```
print_r($lista);
```

Para recorrer un array sin utilizar índices

- **current**- devuelve el valor del elemento que indica el puntero
- **pos** - realiza la misma función que **current**

- **reset**- mueve el puntero al **primer** elemento de la tabla
- **end**- mueve el puntero al **último** elemento de la tabla
- **next**- mueve el puntero al elemento **siguiente**
- **prev**- mueve el puntero al elemento **anterior**
- **count** - devuelve el número de elementos de una tabla.

Ejemplo 3: "Mi primer array"

```
<?php
$semana = array("lunes", "martes", "miércoles", "jueves", "viernes", "sábado",
"domingo");
echo count($semana);    // muestra 7
reset($semana);         //situamos el puntero en el primer elemento
echo current($semana);  // muestra lunes
next($semana);
echo pos($semana);      //martes
end($semana)
echo pos($semana);      //domingo
prev($semana);
echo current($semana);  //sábado
?>
```

Un elemento de un array puede ser a su vez otro array, podemos construir arrays multidimensionales.

```
$lista=array('pepe','juan');
$notas=array(6,7);
$clase=array($lista,$notas);
```

Otra sintaxis

```
$clase=array(array('pepe','juan'),array(6,7));
```

En el caso de los arrays multidimensionales, para referirnos a un dato concreto tendremos que concatenar sus índices entre corchetes.

```
$clase[1][1].
```

Ejemplo 4: "arrays asociativos"

```
<?php
$pais=array (
    "espana"=>array (
        "nombre"=>"España",
        "lengua"=>"Castellano",
        "monedaAntigua"=>"Peseta" ),

    "francia"=>array
        (
            "nombre"=>"Francia",
            "lengua"=>"Francés",
            "monedaAntigua"=>"Franco" )
    );
echo $pais["espana"]["monedaAntigua"]
?>
```